LabFlow Deck - Mowgli Spec (v8)

LabFlow Deck - Premium mobile command center for AI-assisted development.

LabFlow Deck is a mobile-first Progressive Web App (PWA) optimized for iPhone, designed as a premium, single-user control center for AI-assisted software development. It serves solo builders who need to manage GitHub-linked coding projects, GitHub Copilot-powered agent workflows, remote terminal environments, and Docker/Docker Compose deployments on their own servers or personal PCs—all from their phone.

Product Principles

- **Mobile-first, not desktop-shrunk**: Every interaction, layout, and decision prioritizes iPhone usability with one-handed operation
- **Control plane, not compute plane**: The app manages, monitors, and orchestrates—it doesn't try to be the IDE or terminal itself
- **Safe by default**: Risky actions require explicit confirmation; destructive operations are gated and intentional
- **Fast-glance status first**: Critical health information is immediately visible without deep navigation
- **Guided complexity**: Simple tasks are simple; advanced power is available through progressive disclosure
- **Progressive disclosure**: Show what users need now, reveal depth on demand
- **Operational clarity over terminal fidelity**: Clear status indicators and actionable insights over raw terminal output

Non-Goals for v1

The following features are explicitly out of scope for the initial release:

- Team collaboration or shared workspaces
- Role-based access control (RBAC) or permissions systems
- Kubernetes support or orchestration
- Production multi-cloud orchestration
- Full desktop shell parity or complete terminal emulation
- Complex event-driven workflow automation
- Template marketplace or sharing workflows between users
- Deep multi-provider AI abstraction (GitHub Copilot is the primary provider)
- Full code-editing IDE behavior or in-app code modification

User Journeys

1. First-Time Setup and Onboarding

1.1. Welcome and GitHub Authentication

- 1.1.1. User opens LabFlow Deck PWA on iPhone and sees welcome screen with value proposition

- 1.1.2. User initiates GitHub authorization flow (OAuth)—this is the only authentication method for v1
- 1.1.3. System requests repository access permissions (selective deep integration)
- 1.1.4. User completes GitHub connection and returns to app
- 1.1.5. System confirms GitHub Copilot provider status (connected/pending configuration)
- 1.1.6. GitHub identity becomes the user's primary login method

1.2. Host Pairing and Setup

- 1.2.1. User navigates to host onboarding
- 1.2.2. System presents agent-based pairing as the primary method
- 1.2.3. System generates secure pairing command/script for agent-based installation
- 1.2.4. User copies command and runs it on their server/PC (outside app)
- 1.2.5. Host registers with user account via secure token exchange
- 1.2.6. System validates SSH connectivity, Docker availability, and host health
- 1.2.7. Host appears in control panel with status indicator (healthy/degraded/offline)
- 1.2.8. Alternative: User can access manual SSH configuration (advanced fallback) to enter IP, port, user, and key details—this is positioned as an advanced option, not the default

1.3. First Project Creation

- 1.3.1. User prompted to create first project
- 1.3.2. User selects GitHub repository from connected account (repo list fetched)
- 1.3.3. User selects paired host from available hosts list
- 1.3.4. System discovers Docker/Compose configuration files automatically
- 1.3.5. User confirms which services to track in Deploy panel (auto-discovered, manual confirmation)
- 1.3.6. Project created with repository metadata, host link, and deployment targets
- 1.3.7. System presents option to attach workflow now or later (workflows are optional but encouraged)

1.4. Optional First Workflow

- 1.4.1. If user chooses to add workflow, system presents template library (Run Tests, Build Project, Deploy to Staging, etc.)
- 1.4.2. User selects template
- 1.4.3. System pre-populates workflow steps
- 1.4.4. User reviews and customizes steps, commands, and tools
- 1.4.5. User sets execution schedule (cron) or leaves as manual-only
- 1.4.6. Workflow attached to project
- 1.4.7. User completes onboarding and lands on Hub dashboard

2. Daily Monitoring and Quick Actions (Hub)

2.1. Dashboard Overview

- 2.1.1. User opens app and lands on Hub (default tab)
- 2.1.2. System displays overall system health indicator (healthy/degraded/failed)
- 2.1.3. User sees active agent workflows list with current status (running/completed/failed)

- 2.1.4. User sees connected hosts status cards (online/offline/attention needed)
- 2.1.5. User sees deployment status summary (running/stopped/unhealthy)
- 2.1.6. User sees GitHub Copilot provider connection status
- 2.1.7. User sees recent activity feed (scrollable, time-stamped)
- 2.1.8. User sees critical alerts section (failures, completions, offline hosts) with clear status badges

2.2. Status Updates and Refresh

- 2.2.1. User pulls down to refresh Hub data manually
- 2.2.2. System auto-refreshes data lightly while user actively views screen
- 2.2.3. User receives push notification for critical events (workflow failure, deploy failure, host offline, container unhealthy)
- 2.2.4. User taps notification to navigate to relevant detail view

2.3. Quick Actions

- 2.3.1. User taps quick action on workflow card to trigger manual run
- 2.3.2. User taps quick action on deployment card to restart or view logs—with explicit confirmation for restart
- 2.3.3. User taps alert to view details and dismiss or act

3. Project Management

3.1. Projects List

- 3.1.1. User navigates to Projects tab
- 3.1.2. System displays list of project cards
- 3.1.3. Each card shows: project name, repository identifier, host status, workflow status (if attached), last run status, deployment status
- 3.1.4. User scrolls through stacked cards
- 3.1.5. User taps "Create New Project" to start creation flow

3.2. Project Creation Flow

- 3.2.1. User enters project name
- 3.2.2. User selects GitHub repository from connected account (file tree and metadata visible)
- 3.2.3. User selects remote host from paired hosts
- 3.2.4. System auto-detects Docker/Compose services and presents for confirmation
- 3.2.5. User selects which services to track (can rename/label for clarity)
- 3.2.6. User optionally attaches existing workflow or creates new one (workflows are optional)
- 3.2.7. Project created and added to list

3.3. Project Detail Navigation

- 3.3.1. User taps project card to open Project Detail
- 3.3.2. System loads project overview with sections: Repository, Terminal Runtime, Workflow, Deployments, Logs, Stats
- 3.3.3. User navigates between sections via segmented control or scrollable areas

4. Project Detail Operations

4.1. Repository Section

- 4.1.1. User views repository metadata (name, branch, last commit)
- 4.1.2. User browses file tree structure (fetched on-demand)
- 4.1.3. User taps file to view contents (source, config, documentation) for workflow context (fetched on-demand, no full repo sync)
- 4.1.4. User sees branch selector

4.2. Terminal Runtime Section

- 4.2.1. User views live log viewer showing recent output from host
- 4.2.2. User sees predefined project actions (start, stop, restart, rebuild, inspect, validate)—these are the primary interaction method
- 4.2.3. User taps action to execute structured command—with explicit confirmation for risky actions (stop, restart, rebuild)
- 4.2.4. System displays command output in scrollable, readable log format (all output logged to ExecutionLog)
- 4.2.5. User accesses recent command history
- 4.2.6. User enters lightweight custom command via input field (secondary use case)—dangerous commands require confirmation
- 4.2.7. User sees session status and failure feedback

4.3. Workflow Assignment Section

- 4.3.1. User sees currently attached workflow (if any) with last run status—projects can exist without workflows
- 4.3.2. User taps to change workflow or attach new one (workflows can be reused across projects)
- 4.3.3. User views upcoming scheduled runs (if schedule configured)
- 4.3.4. User views recent run history with success/failure status
- 4.3.5. User triggers manual run from this section
- 4.3.6. User pauses, resumes, or disables schedule

4.4. Deployments Section

- 4.4.1. User sees list of tracked Docker/Compose services
- 4.4.2. Each service shows runtime status badge (running, stopped, restarting, failed, deploying), health status badge (healthy, unhealthy, unknown), resource usage, and health indicator
- 4.4.3. User taps service to view detailed status and recent logs
- 4.4.4. User executes actions: deploy, rebuild, restart, stop—with explicit confirmation step for all risky actions
- 4.4.5. User sees deployment preview/status URLs if configured

4.5. Logs and Stats Sections

- 4.5.1. User views human-readable activity timeline (ActivityLog) showing system events
- 4.5.2. User views raw execution output (ExecutionLog) from workflows, deployments, terminal, and host agent

- 4.5.3. User filters logs by source (workflow, deployment, terminal, host_agent) and severity
- 4.5.4. User pins important logs for retention
- 4.5.5. User exports logs before deletion
- 4.5.6. User views project statistics: build times, success rates, resource usage

5. Agent Workflow Management

5.1. Workflow List

- 5.1.1. User navigates to Agents tab
- 5.1.2. System displays list of existing workflows
- 5.1.3. Each workflow shows name, template source (if any), schedule status, last run status, and which projects use it
- 5.1.4. User taps workflow to edit or run

5.2. Create Workflow

- 5.2.1. User taps "Create Workflow"
- 5.2.2. System presents template library (Run Tests, Build Project, Deploy to Staging, Inspect Repository, Validate Deployment, Restart Service, Summarize Logs)
- 5.2.3. User selects template or chooses "Start from Scratch" (advanced)
- 5.2.4. System loads workflow editor with steps pre-populated (if template) or empty (if scratch)

5.3. Workflow Editor

- 5.3.1. User sees ordered list of workflow steps
- 5.3.2. User adds, removes, or reorders steps
- 5.3.3. User defines step type: command execution, file operation, AI analysis, validation, notification
- 5.3.4. User assigns tools/skills/commands per step
- 5.3.5. User configures step parameters and failure handling
- 5.3.6. User sets workflow schedule (cron expression) or leaves as manual-only
- 5.3.7. User saves workflow

5.4. Test and Run

- 5.4.1. User triggers test run from workflow view
- 5.4.2. System executes workflow on selected project/host
- 5.4.3. User sees real-time execution status and step progress
- 5.4.4. User views execution results and output logs
- 5.4.5. User reviews run history with timestamps and status
- 5.4.6. User views upcoming scheduled runs in calendar/list format

6. Deployment Management

6.1. Deploy Panel Overview

- 6.1.1. User navigates to Deploy tab
- 6.1.2. System displays deployment cards for all projects
- 6.1.3. Each card shows: project name, service count, overall status (runtime + health),

resource usage summary
- 6.1.4. User filters by project or status

6.2. Service Management

- 6.2.1. User taps deployment card to see service list
- 6.2.2. User sees individual service cards with: name, runtime status badge, health status badge, resource metrics, recent log preview
- 6.2.3. User taps service for detailed view
- 6.2.4. User executes actions: deploy, rebuild, restart, stop—with explicit confirmation dialogs for all destructive actions
- 6.2.5. User inspects full logs in scrollable viewer
- 6.2.6. User views health checks and preview URLs

7. Host Management

7.1. Host List and Access

- 7.1.1. User accesses hosts via Settings or Hub dashboard
- 7.1.2. System displays list of paired hosts with connection status, registration status, and capabilities
- 7.1.3. User taps host to open Host Detail (first-class screen)
- 7.1.4. User adds new host via agent pairing flow (primary method) or manual SSH configuration (advanced fallback)

7.2. Host Detail Screen

- 7.2.1. User views host header with name, connection mode (agent/ssh_manual), and registration status
- 7.2.2. System displays system info card with OS details, agent version, Docker/Compose versions
- 7.2.3. User views connection health with status, latency, last heartbeat, and public key fingerprint
- 7.2.4. User sees resource usage metrics (CPU, memory, disk) with visual indicators
- 7.2.5. User views host agent logs (filtered by severity)
- 7.2.6. User sees list of associated projects with quick links to project details
- 7.2.7. User executes host actions: reconnect, edit configuration, remove host (with explicit confirmation)

8. Insights and Analytics

8.1. Insights Dashboard

- 8.1.1. User navigates to Insights from Hub or Projects
- 8.1.2. System displays workflow success rate card
- 8.1.3. System displays deployment success rate card
- 8.1.4. System displays average run time metrics
- 8.1.5. System displays most active projects list
- 8.1.6. System displays host uptime percentage
- 8.1.7. System displays failure trends over time
- 8.1.8. User views data in clean charts and readable cards optimized for small screens

9. Settings and Configuration

9.1. Settings Navigation

- 9.1.1. User accesses Settings via profile/menu route (not bottom tab)
- 9.1.2. User sees settings categories: GitHub Connection, AI Provider, Hosts, Deployment Defaults, Notifications, Security, Log Retention

9.2. GitHub and Provider Configuration

- 9.2.1. User views GitHub connection status and permissions
- 9.2.2. User configures GitHub Copilot integration settings
- 9.2.3. User manages repository access scope (selective integration)
- 9.2.4. User can disconnect GitHub—with explicit confirmation dialog

9.3. Host Management

- 9.3.1. User views list of paired hosts with connection mode and registration status
- 9.3.2. User adds new host via agent pairing flow (primary)
- 9.3.3. User edits host details (name, Tailscale IP)
- 9.3.4. User accesses manual SSH configuration for advanced setup (fallback)
- 9.3.5. User removes host—with explicit confirmation

9.4. System Preferences

- 9.4.1. User configures notification preferences (push, in-app) for critical events
- 9.4.2. User sets log retention policy (default 14-30 days auto-purge, customizable)
- 9.4.3. User configures deployment defaults (auto-restart, health check intervals)
- 9.4.4. User manages security preferences (session timeout, biometric lock if supported)

Data Model

User

Represents the single account holder. GitHub is the primary identity provider for v1.

Fields:

- `id`: Unique identifier
- `githubUserId`: GitHub user ID
- `githubAccessToken`: Encrypted GitHub OAuth token (primary auth method)
- `githubUsername`: GitHub username
- `githubAvatarUrl`: Profile image URL
- `copilotConfig`: JSON configuration for GitHub Copilot integration
- `notificationPreferences`: JSON (push enabled, in-app enabled, critical alerts only)
- `createdAt`: Timestamp
- `updatedAt`: Timestamp

Relationships:

- Has many `Host` entities

- Has many `Project` entities
- Has many `Workflow` entities
- Has many `ActivityLog` entities
- Has many `ExecutionLog` entities
- Has many `Notification` entities
- Has one `Settings` entity

Host

Represents a user-owned server or PC connected via agent-based pairing (primary) or manual SSH (fallback).

Fields:

- `id`: Unique identifier
- `userId`: FK User
- `name`: Display name (e.g., "Home Server")
- `connectionMode`: Enum [`agent`, `ssh_manual`] (agent is primary)
- `registrationStatus`: Enum [`pending`, `active`, `failed`, `revoked`]
- `pairingToken`: Secure token for agent-based registration (agent mode only)
- `agentVersion`: Version string of installed agent (agent mode only)
- `publicKeyFingerprint`: Fingerprint for trust verification
- `capabilities`: JSON array of detected capabilities (`docker`, `compose`, `logs`, `metrics`, `terminal_actions`)
- `manualSshConfigEncrypted`: Encrypted SSH config (ssh_manual mode only, advanced fallback)
    - `sshHost`: IP address or hostname
    - `sshPort`: Port number (default 22)
    - `sshUser`: Username
    - `sshKey`: Encrypted private key
- `tailscaleIp`: Optional Tailscale IP address
- `status`: Enum [`healthy`, `degraded`, `offline`, `unavailable`]
- `dockerVersion`: Detected Docker version
- `composeVersion`: Detected Compose version
- `osInfo`: Operating system details
- `lastSeen`: Timestamp of last health check
- `createdAt`: Timestamp

Relationships:

- Belongs to `User`
- Has many `Project` entities
- Has many `Deployment` entities

Project

Represents a GitHub-linked coding project with optional workflow attachment.

Fields:

- `id`: Unique identifier
- `userId`: FK User

- `name`: Project display name
- `githubRepoUrl`: Full repository URL
- `githubRepoFullName`: Owner/Repo format
- `defaultBranch`: Default branch name
- `hostId`: FK Host (required)
- `workflowId`: FK Workflow (nullable, optional for v1)
- `composeFilePath`: Path to docker-compose.yml or equivalent
- `createdAt`: Timestamp
- `updatedAt`: Timestamp

Relationships:

- Belongs to `User`
- Belongs to `Host`
- Belongs to `Workflow` (optional, can be shared across projects)
- Has many `Deployment` entities
- Has many `WorkflowRun` entities (through Workflow)
- Has many `ActivityLog` entities

Workflow

Represents an AI-powered agent workflow with steps and scheduling. Can be reused across multiple projects.

Fields:

- `id`: Unique identifier
- `userId`: FK User
- `name`: Workflow name
- `description`: Optional description
- `templateSource`: Enum [`custom`, `run_tests`, `build_project`, `deploy_staging`, `inspect_repo`, `validate_deploy`, `restart_service`, `summarize_logs`]
- `steps`: JSON array of workflow steps (ordered, with commands, tools, parameters)
- `schedule`: Optional cron expression for scheduled runs
- `isActive`: Boolean (schedule enabled/disabled)
- `createdAt`: Timestamp
- `updatedAt`: Timestamp

Relationships:

- Belongs to `User`
- Has many `WorkflowRun` entities
- Has many `Project` entities (assigned to, one per project in v1)

WorkflowRun

Represents a single execution of a workflow.

Fields:

- `id`: Unique identifier
- `workflowId`: FK Workflow
- `projectId`: FK Project (context of execution)

- `status`: Enum [`pending`, `running`, `completed`, `failed`, `cancelled`]
- `triggeredBy`: Enum [`manual`, `scheduled`]
- `startedAt`: Timestamp
- `completedAt`: Timestamp
- `executionLogId`: FK ExecutionLog (raw output)
- `output`: JSON structured results
- `errorMessage`: Failure details if failed

Relationships:

- Belongs to `Workflow`
- Belongs to `Project`
- Has one `ExecutionLog`

Deployment

Represents Docker/Compose deployment configuration for a project.

Fields:

- `id`: Unique identifier
- `projectId`: FK Project
- `hostId`: FK Host
- `composeConfig`: JSON parsed compose configuration
- `runtimeStatus`: Enum [`active`, `stopped`, `error`, `deploying`]
- `createdAt`: Timestamp
- `updatedAt`: Timestamp

Relationships:

- Belongs to `Project`
- Belongs to `Host`
- Has many `DeploymentService` entities

DeploymentService

Represents individual services/containers within a deployment.

Fields:

- `id`: Unique identifier
- `deploymentId`: FK Deployment
- `name`: Service name
- `containerId`: Docker container ID
- `image`: Docker image name
- `runtimeStatus`: Enum [`running`, `stopped`, `restarting`, `failed`, `deploying`]
- `healthStatus`: Enum [`healthy`, `unhealthy`, `unknown`]
- `healthCheckResult`: JSON with detailed health check output
- `resourceUsage`: JSON (CPU, memory, network stats)
- `ports`: JSON exposed ports
- `lastUpdated`: Timestamp

Relationships:

- Belongs to `Deployment`
- Has many `ExecutionLog` entities

ActivityLog

Represents human-readable timeline of system events and user actions.

Fields:

- `id`: Unique identifier
- `userId`: FK User
- `projectId`: Optional FK Project
- `type`: Enum [`workflow_started`, `workflow_completed`, `workflow_failed`, `deploy_started`, `deploy_completed`, `deploy_failed`, `host_offline`, `host_recovered`, `terminal_command`, `settings_changed`, `host_added`, `host_removed`, `project_created`]
- `message`: Human-readable description
- `metadata`: JSON additional context
- `severity`: Enum [`info`, `warning`, `critical`]
- `createdAt`: Timestamp

Relationships:

- Belongs to `User`
- Belongs to `Project` (optional)

ExecutionLog

Represents raw execution output from various sources.

Fields:

- `id`: Unique identifier
- `sourceType`: Enum [`workflow_run`, `deployment_service`, `terminal_command`, `host_agent`]
- `sourceId`: ID of the source entity (WorkflowRun.id, DeploymentService.id, etc.)
- `projectId`: FK Project (for context)
- `content`: Raw log output text
- `severity`: Enum [`debug`, `info`, `warning`, `error`]
- `retentionClass`: Enum [`short`, `medium`, `long`] (for auto-purge policy)
- `isPinned`: Boolean (user-pinned for retention)
- `createdAt`: Timestamp

Relationships:

- Belongs to `Project`

Notification

Represents push/in-app notifications for critical events.

Fields:

- `id`: Unique identifier

- `userId`: FK User
- `type`: Enum [`workflow_failure`, `workflow_completion`, `deploy_failure`, `deploy_completion`, `host_offline`, `container_unhealthy`, `provider_auth_error`]
- `title`: Notification headline
- `body`: Notification content
- `channel`: Enum [`push`, `in_app`]
- `deliveryStatus`: Enum [`pending`, `sent`, `failed`, `dismissed`]
- `isRead`: Boolean
- `readAt`: Timestamp (nullable)
- `clickedAt`: Timestamp (nullable)
- `actionUrl`: Deep link to relevant screen
- `createdAt`: Timestamp

Relationships:

- Belongs to `User`

Settings

User preferences and system configuration.

Fields:

- `id`: Unique identifier
- `userId`: FK User
- `logRetentionDays`: Integer (default 30)
- `autoPurgeEnabled`: Boolean
- `defaultNotificationPrefs`: JSON (push critical only, etc.)
- `securitySettings`: JSON (biometric lock, session timeout)
- `deploymentDefaults`: JSON (auto-restart policies, health check intervals)

Relationships:

- Belongs to `User`

Design

The frontend should implement a premium dark command-center aesthetic optimized for iPhone.

Color Palette:

- Obsidian Navy #0B1020 (primary background)
- Graphite #141B2D (secondary background/surfaces)
- Slate Surface #1E293B (tertiary surfaces, cards)
- Electric Blue #3B82F6 (primary accent, interactive elements)
- Cyan Glow #22D3EE (secondary accent, status indicators)
- Soft Violet #8B5CF6 (tertiary accent, highlights)
- Success Green #22C55E (healthy/running status)
- Warning Amber #F59E0B (warning/degraded status)
- Alert Red #EF4444 (failed/error status)
- Primary Text #E5EEF9 (headings, primary content)

Visual Direction:

- iPhone-first layout and spacing with one-handed usability
- Premium, minimal, modern interface
- Strong hierarchy with large headings and clean card-based sections
- Rounded panels and refined spacing
- Compact but readable metrics and status cards
- Smooth, polished interactions
- Subtle depth, glow, or glass-like accents used carefully
- Professional, not playful
- Stacked cards over dense tables
- Clear status badges (healthy, running, degraded, failed, offline, attention needed)
- Clean logs and activity views
- Fast-glance readability
- Sleek, high-end, technical, and modern with subtle glow accents and strong contrast for mobile readability
- Explicit confirmation dialogs for all risky actions
- Guided empty states that teach the next action

Frontend

Primary Navigation (Mobile)

- Fixed bottom tab bar with four primary tabs: Hub, Projects, Agents, Deploy
- Large tap targets optimized for thumb reach
- Active tab indicator using Electric Blue or Cyan Glow
- Profile/Menu access via top-right avatar or hamburger (Settings route)

Profile Menu

- Triggered by tapping the top-right avatar
- Provides access to Settings, Hosts list, and user profile summary
- Displays as a bottom sheet or modal overlay

Status Indicators

- Health badges using Success Green, Warning Amber, Alert Red, and Electric Blue
- Connection status dots for hosts and providers
- Activity pulse indicators for running workflows
- Resource usage mini-bars in deployment cards
- Separate runtime status and health status badges for deployment services

Notification System

- In-app notification center accessible from Hub top-right
- Push notification support for critical events only
- Toast alerts for action confirmations (success/error)
- Pull-to-refresh gesture on scrollable lists
- Notification delivery status indicators

Confirmation Dialogs

- Explicit confirmation required for all risky actions:
  - Restart service
  - Stop service
  - Rebuild service
  - Redeploy
  - Remove host
  - Disconnect GitHub
- Clear messaging about what will happen
- Confirmation button styled distinctly from cancel

Empty States

- Strong guided empty states across all major screens:
  - "Connect GitHub to Get Started" when no GitHub connected
  - "Pair Your First Host" when no hosts available
  - "Create Your First Project" when no projects
  - "Create a Workflow" when no workflows
  - "No Active Deployments" when no deployments
  - "No Alerts" when no alerts
  - "No Logs Yet" when no logs
  - "Insights Will Appear Here" when no data yet
- Each empty state includes:
  - Clear illustration or icon
  - Brief explanation of why this is empty
  - Primary action button to resolve

OnboardingScreen

Summary: Multi-step onboarding flow for new users, covering authentication, host setup, and initial project configuration.

Preview size: 390x844

Preview states

State	Name	Description
ID: welcome	Welcome Screen	The initial screen displayed to new users with the value proposition and a "Get Started" button.
ID: githubAuthPrompt	GitHub Auth Prompt	Screen prompting the user to authenticate via GitHub OAuth, explaining it is the only auth method.

ID: hostPrompt	Host Onboarding Prompt	Screen asking the user to pair their first host, highlighting agent-based pairing as the primary method.
ID: hostPairing	Host Pairing Command	Screen displaying the generated secure pairing command/script with a "Copy" button and optional QR code for the user to run on their server.
ID: hostValidation	Host Validation	Loading state showing the system validating SSH connectivity, Docker availability, and host health after the pairing command was run.
ID: createProjectIntro	First Project Creation Intro	Introductory step for creating the first project, prompting for name and basic info.
ID: createProjectRepo	Select GitHub Repo	Step where the user selects a GitHub repository from their connected account.
ID: createProjectHost	Select Host	Step where the user selects the paired host to use for the project.
ID: createProjectServices	Confirm Services	Step showing auto-detected Docker/ Compose services with toggles for the user to confirm which to track.
ID: createProjectWorkflow	Attach Workflow	Optional step allowing the user to attach an existing workflow or create a new one.

Contents

The initial entry flow for new users covering welcome, GitHub authentication, host setup, and first project creation.

Content Hierarchy:

- **Welcome Screen**: Value proposition, "Get Started" button, and brand presentation.
- **GitHub Auth**: External OAuth flow (only authentication method for v1).
- **Host Onboarding Prompt**: Prompt to pair a user-owned server/PC via agent-based pairing.
- **Host Pairing**: Display of generated secure pairing command with "Copy" button and optional QR code.
- **Host Validation**: Loading state indicating validation of SSH, Docker, and Compose availability.
- **First Project Creation**: Multi-step flow to define name, repo, host, services, and optional workflow.

Nested Elements:

- **Manual SSH Fallback**: Advanced option for users who prefer manual configuration (not prominently featured).
- **Create Project Flow**: Modal or full-screen sequence (Name/Repo, Host, Services, Workflow).

HubScreen

Summary: The main dashboard displaying system health, active workflows, host status, deployments, and recent activity.

Preview size: 390x844

Preview states

State	Name	Description
ID: default	Default Healthy State	The dashboard view with all systems healthy, showing recent activity and running workflows.
ID: degradedState	Degraded Status State	Dashboard view where one or more elements (host, deployment) are in a degraded or warning state (Amber badge).
ID: criticalAlert	Critical Alert Visible	Dashboard view with a red Alert Banner visible at the top, indicating a critical failure (e.g., host offline).
ID: notificationCenter	Notification Center Open	A full-screen overlay or modal showing the history of critical alerts with dismiss actions.
ID: activityDetail	Activity Detail Sheet Open	A bottom sheet expanded from the activity feed, showing detailed metadata and a logs preview for a specific event.
ID: quickActionMenu	Quick Action Menu Open	A bottom sheet menu presenting quick action buttons for triggering common workflows or deployments.

Contents

Mission-control dashboard communicating system health in seconds.

Content Hierarchy:

- Overall system health indicator (large status badge)
- Active agent workflows section (horizontal scroll or stacked cards showing running/completed/failed)
- Connected hosts status grid (host cards with online/offline indicators and connection mode)

- Deployment status summary (count of healthy vs. total services, separated runtime and health)
- Provider connection status card (GitHub Copilot)
- Critical alerts section (expandable, prioritized by severity)
- Recent activity feed (vertical list, time-stamped, scrollable, human-readable)
- Quick action buttons for common operations (with confirmation where needed)

Nested Elements:

- **Notification Center**: Full-screen overlay or modal showing critical alerts history with dismiss actions and read status.
- **Activity Detail Bottom Sheet**: Expanded view of specific activity item with metadata and logs preview.
- **Quick Action Menu**: Bottom sheet for triggering common workflows or deployments.
- **Alert Banner**: Red banner appearing for critical status changes (e.g., Host Offline).

ProjectsScreen

Summary: List view of all projects with options to filter, create new ones, and manage existing projects.

Preview size: 390x844

Preview states

State	Name	Description
ID: default	Project List	The main view showing a list of stacked project cards with their status indicators.
ID: empty	Empty State	View displayed when no projects exist, showing a guided empty state with a "Create Your First Project" call to action.
ID: createStep1	Create Project: Name Repo	First step of the create flow modal, asking for project name and GitHub repo selection.
ID: createStep2	Create Project: Select Host	Second step of the create flow modal, asking to select a host from paired hosts.
ID: createStep3	Create Project: Confirm Services	Third step of the create flow modal, displaying auto-detected Docker services for confirmation.
ID: createStep4	Create Project: Attach Workflow	Final step of the create flow modal, offering the option to attach a workflow (marked as optional).
ID: contextMenu	Project Context Menu	A menu (bottom sheet or popover) triggered by a swipe or long-press on a project card, offering Edit, Duplicate, or Delete options.

ID: deleteConfirm	Delete Confirmation Dialog	A confirmation dialog warning the user about the deletion of a project.

Contents

Project management panel for viewing and creating projects.

Content Hierarchy:

- Header with title and "Create Project" primary action
- Filter/sort controls (by status, last active)
- Stacked project cards list
- Each card displays: project name, repository identifier, host status dot, workflow status (if attached—"Optional" badge if none), last run status badge, deployment status summary
- Swipe actions on cards for quick edit or archive
- Empty state with guided onboarding when no projects

Nested Elements:

- **Create Project Flow**: Multi-step modal/bottom sheet sequence:
  - Step 1: Project name and GitHub repo selection (with repo browser, on-demand fetch)
  - Step 2: Host selection from paired hosts
  - Step 3: Service discovery confirmation (auto-detected Docker services with toggle selection)
  - Step 4: Optional workflow attachment (clearly marked as optional)
- **Project Context Menu**: Options to edit, duplicate, or delete project (with explicit confirmation for deletion)

ProjectDetailScreen

Summary: Detailed view of a single project containing tabs for Repository, Terminal, Workflows, Deployments, Logs, and Stats.

Preview size: 390x844

Preview states

State	Name	Description
ID: overview	Overview Tab	The default view showing quick stats, recent activity, and general project information.
ID: repository	Repository Tab	View showing the branch selector and file tree browser.

ID: fileViewer	File Viewer Modal	Full-screen modal displaying the content of a selected file with syntax highlighting.
ID: terminal	Terminal Tab	View showing the live log viewer and the predefined action grid (start, stop, restart, etc.).
ID: terminalConfirm	Terminal Action Confirmation	A confirmation dialog visible over the terminal tab, asking for explicit confirmation before a risky action like "Restart".
ID: workflow	Workflow Tab	View showing the attached workflow card, schedule status, upcoming runs, and history.
ID: workflowSelector	Workflow Selector	A bottom sheet menu allowing the user to attach or change the currently assigned workflow.
ID: deploy	Deploy Tab	View listing the tracked services with their runtime status, health status, and resource usage.
ID: deployActionConfirm	Deploy Action Confirmation	A confirmation dialog for a destructive deployment action (e.g., Stop Service).
ID: logs	Logs Tab	View showing the aggregated logs with filter tabs (Activity vs Execution).
ID: logsFilter	Logs Filter Menu	A menu open to filter logs by source, time range, or severity.
ID: stats	Stats Tab	View displaying charts for build times, success rates, and resource usage.

Contents

Detailed project screen with multiple functional sections.

Content Hierarchy:

- Project header with name, repo link, and status
- Segmented control or tab navigation for sections: Overview, Repository, Terminal, Workflow, Deploy, Logs, Stats
- **Overview Section**: Quick stats, recent activity, action buttons
- **Repository Section**: Branch selector, file tree browser (fetched on-demand), file content viewer for selected files (no full repo sync)
- **Terminal Runtime Section**: Live log viewer (scrollable), predefined action grid (start,

stop, restart, rebuild, inspect, validate)—primary interaction, command history list, lightweight command input field (secondary), all risky actions require confirmation
- **Workflow Section**: Currently attached workflow card (shows "No workflow attached" if none), schedule status toggle, upcoming runs list, recent run history with status, manual run trigger, option to attach workflow
- **Deploy Section**: Service cards list with separate runtime status and health status badges, resource usage mini-charts
- **Logs Section**: Aggregated logs viewer with source filter tabs (ActivityLog vs ExecutionLog), pin/export actions
- **Stats Section**: Build time trends, success rate charts, resource usage over time

Nested Elements:

- **File Viewer Modal**: Full-screen code/config file view with syntax highlighting (fetched on-demand).
- **Command Output Sheet**: Expanded log view for command execution results (all logged).
- **Confirmation Dialog**: Explicit confirmation for stop, restart, rebuild actions.
- **Workflow Selector**: Bottom sheet to attach or change workflow (shows reusable workflows).
- **Service Action Confirmation**: Alert dialog for destructive actions (stop, rebuild) requiring explicit confirmation.
- **Log Filter Menu**: Options to filter by source (workflow, deployment, terminal, host_agent), time range, severity.

HostDetailScreen

Summary: Management and diagnostic screen for a specific remote host, showing system info, resource usage, and connection health.

Preview size: 390x844

Preview states

State	Name	Description
ID: default	Host Overview	The main view displaying system info, connection health, resource usage charts, agent logs, and associated projects.
ID: resourceDetail	Resource Detail Sheet	A bottom sheet or modal showing an expanded history view of CPU, Memory, and Disk usage.
ID: editModal	Edit Host Configuration	A modal form allowing the user to update the host name, Tailscale IP, or override SSH settings.
ID: removeConfirm	Remove Host Confirmation	A confirmation dialog warning the user about the removal of the host and its impact on projects.

Contents

First-class diagnostic and management screen for a specific remote host. This is a primary screen in the product architecture.

Content Hierarchy:

- Host header with name, connection mode badge (agent/ssh_manual), and registration status
- **System Info Card**: OS details, CPU/RAM overview (if available), Docker version, Compose version, agent version (if applicable)
- **Connection Health**: Status of SSH connection, Tailscale IP (if configured), last heartbeat timestamp, latency info, public key fingerprint
- **Resource Usage**: Host-level resource metrics (CPU %, Memory %, Disk usage) visualized with compact charts
- **Agent Logs**: Live log viewer for the connection agent and host-side daemon logs (scrollable, filtered by severity)
- **Associated Projects**: List of projects running on this host with quick links to ProjectDetail
- **Actions**: "Reconnect Host" button, "Edit Configuration", "Remove Host" (with explicit confirmation)

Nested Elements:

- **Resource Detail Sheet**: Expanded view of CPU, Memory, and Disk history.
- **Agent Log Filter**: Filter logs by severity (Info, Warning, Error).
- **Host Edit Modal**: Form to update host name, Tailscale IP, or manually override SSH settings.
- **Host Action Menu**: Options to edit or remove host (triggered from settings or detail view).
- **Capability Badges**: Visual indicators of detected capabilities (docker, compose, logs, metrics, terminal_actions).

AgentWorkflowPanelScreen

Summary: Panel for managing, creating, editing, and testing AI-powered agent workflows.

Preview size: 390x844

Preview states

State	Name	Description
ID: default	Workflow List	The list view of all saved workflows with their status and project counts.
ID: templateGallery	Template Gallery	A grid or list view of available workflow templates shown when creating a new workflow.
ID: editor	Workflow Editor	The editor view displaying the ordered list of steps, configuration panel, and schedule settings.

ID: stepEditor	Step Editor Sheet	A bottom sheet open for configuring the details of an individual workflow step (command, tools).
ID: testRunMonitor	Test Run Monitor	A real-time view monitoring the execution of a test run, showing step progress and live logs.
ID: runDetail	Run Detail View	A detailed view of a past workflow run, including results, output logs, and error details.

Contents

Workflow builder and testing area.

Content Hierarchy:

- Workflow list with status indicators, schedule icons, and project count
- "Create Workflow" primary action
- Workflow editor view (when creating/editing):
  - Step list (ordered, draggable or manual reorder)
  - Step configuration panel (command, tool, parameters)
  - Schedule configuration (cron input, active toggle)
- Test run section with execution monitor
- Run history list with filter options

Nested Elements:

- **Workflow Template Gallery**: Grid or list view of available templates (Run Tests, Build, Deploy, etc.) presented upon creating a new workflow.
- **Workflow Template Preview**: Description and preview of template steps before selection.
- **Step Editor Sheet**: Bottom sheet for configuring individual step details.
- **Test Run Monitor**: Real-time execution view with step progress indicators and live log output.
- **Schedule Picker**: Interface for setting cron expressions with common presets (hourly, daily, weekly).
- **Run Detail View**: Full execution results, output logs, and error details.
- **Project Assignment**: Shows which projects use this workflow.

DeployPanelScreen

Summary: Management panel for Docker/Compose deployments across all projects.

Preview size: 390x844

Preview states

State	Name	Description

ID: default	Deployment List	List view of deployment cards grouped by project, showing aggregate status and resource usage.
ID: serviceDetail	Service Detail View	Expanded view of a specific service showing status badges, resource metrics, port mappings, and a log preview.
ID: serviceLogs	Service Logs Modal	Full-screen modal showing detailed logs for the selected service.
ID: deployConfirm	Deploy Confirmation Dialog	A confirmation dialog showing a summary of changes before proceeding with a deploy or rebuild.
ID: resourceDetail	Resource Detail Sheet	A bottom sheet showing expanded resource metrics and health check details for a service.
ID: actionConfirm	Service Action Confirmation	A confirmation dialog for a destructive action like Stop or Restart.

Contents

Deployment management for Docker/Compose environments.

Content Hierarchy:

- Deployment cards list (by project)
- Each card shows: project name, environment status, service count, aggregate resource usage, separate runtime and health indicators
- Service detail view (when expanded or tapped):
  - Service name
  - Runtime status badge (running, stopped, restarting, failed, deploying)
  - Health status badge (healthy, unhealthy, unknown)
  - Resource usage metrics (CPU, memory)
  - Port mappings
  - Recent log preview
- Action toolbar: Deploy, Rebuild, Restart, Stop (all with explicit confirmation)
- Log viewer for selected service

Nested Elements:

- **Service Logs Modal**: Full-screen log viewer with search and filter.
- **Deploy Confirmation Dialog**: Summary of changes before deploy/rebuild.
- **Resource Detail Sheet**: Expanded metrics and health check details.
- **Environment Variables Viewer**: Read-only view of env vars (if accessible).
- **Confirmation Dialogs**: Explicit confirmation for rebuild, restart, stop actions.

InsightsScreen

Summary: Analytics dashboard displaying success rates, runtime metrics, and failure trends.

Preview size: 390x844

Preview states

State	Name	Description
ID: default	Insights Dashboard	The main view populated with charts for success rates, runtimes, and failure trends.
ID: empty	Empty State	View shown when no data is available yet, displaying a guided empty state message.
ID: chartDetail	Chart Detail View	An expanded view of one of the charts, showing specific data points.
ID: projectDrilldown	Project Activity Drill-down	A detailed view showing specific stats for a project selected from the "Most Active" list.

Contents

Analytics dashboard with clean mobile-optimized charts.

Content Hierarchy:

- Time period selector (7d, 30d, 90d)
- Workflow success rate card (percentage with trend indicator)
- Deployment success rate card
- Average runtime metric card
- Most active projects list (ranked by activity)
- Host uptime percentage card
- Failure trends chart (simple line or bar chart)
- Activity summary cards
- Empty state: "Insights Will Appear Here" with guidance

Nested Elements:

- **Chart Detail View**: Expanded chart with data points.
- **Project Activity Drill-down**: Detailed stats for specific project selected from "Most Active" list.

SettingsScreen

Summary: Settings screen for configuring GitHub, hosts, notifications, security, and app preferences.

Preview size: 390x844

Preview states

State	Name	Description
ID: default	Settings List	The main settings view listing all configuration categories (Profile, GitHub, Hosts, Notifications, etc.).
ID: hostPairingFlow	Host Pairing Flow	A modal flow initiated by "Add Host", displaying the pairing command and QR code.
ID: manualHostConfig	Manual Host Config	A form modal for advanced users to manually configure SSH (IP, port, user, key).
ID: notificationPrefs	Notification Preferences	A bottom sheet or modal with granular toggles for different notification types and channels.
ID: disconnectGithub	Disconnect GitHub Confirmation	A confirmation dialog warning the user about the consequences of disconnecting their GitHub account.

Contents

Configuration panel accessed via profile menu.

Content Hierarchy:

- User profile section (name, GitHub username, avatar)
- GitHub Connection card (status, permissions, disconnect option—with explicit confirmation)
- GitHub Copilot Provider card (configuration status)
- Hosts Management section:
    - List of paired hosts with status, connection mode, registration status
    - Add Host button (triggers agent pairing flow)
    - Manual SSH configuration option (advanced fallback)
- Deployment Defaults section (auto-restart, health check intervals)
- Notifications section (push preferences, in-app preferences, critical alerts toggle)
- Security section (biometric lock, session timeout)
- Log Retention section (retention days setting, auto-purge toggle)
- App Info section (version, legal links)

Nested Elements:

- **Host Pairing Flow**: Step-by-step modal with generated command and QR code option.
- **Manual Host Configuration Form**: Fields for IP, port, user, SSH key input (advanced).
- **SSH Key Management**: View, copy, or regenerate keys (advanced).
- **Confirmation Dialogs**: For disconnecting GitHub (explicit), removing hosts (explicit), or clearing data.

- **Notification Preferences Sheet**: Granular toggles for event types and channels.
- **Host Action Menu**: Options to edit or remove hosts from the host list.
- **Disconnect GitHub Confirmation**: Clear warning about what will happen upon disconnection.